

CODE STRUCTURE PROPOSAL

Streamlit 앱 모듈화 제안

2,245줄에 이르는 `app/streamlit_app.py` 를 어떻게 나누면 학습과 유지보수에 도움이 되는지 정리한 문서입니다. 이번 문서는 제안 단계이며, 실제 파일 분리 작업은 아직 진행하지 않습니다.

목적 — 학습 및 유지보수를 위한 코드 구조 개선 제안 공유

대상 — 팀 전체 (Streamlit 조회 화면 담당자)

현재 단계 — 제안 검토 (실제 분리 작업 미착수)

1 왜 나누려고 하는가

현재 조회 화면은 `app/streamlit_app.py` 한 파일에 모여 있습니다. 대략 **2,245줄**이며, 역할별로 보면 다음과 같습니다.

구간	대략 줄 수	하는 일
CSS	약 780줄	탭, 배너, 카드, 팝업 등 화면 스타일
상수·경로·URL	약 80줄	DB 경로, 제조사 링크, 아이콘 이름
SQL·DB 조회	약 150줄	SQLite 연결, 쿼리, <code>read_query</code>
헬퍼	약 200줄	사진 찾기, 아이콘, 관심 차량 추가·삭제
화면 렌더	약 1,000줄	조회 / 비교 / 도움말 UI
엔트리	약 30줄	공익광고, 헤더, 탭 조립

한 파일에 두면 처음에는 찾기 쉽습니다. 다만 줄 수가 2,000을 넘으면 다음이 어려워집니다.

- 배너 CSS만 고치고 싶은데 비교 화면 코드를 스크롤해야 합니다.
- 팀원이 서로 다른 기능을 고치면 같은 파일에서 충돌이 납니다.
- SQL 문구와 버튼 배치가 섞여 있어 "데이터 문제인지 UI 문제인지" 구분이 어렵습니다.

🔴 판단 기준

Streamlit 공식 안내에 따르면 **1,000줄 이하, 로직이 단순한 앱은 한 파일이 더 낫습니다**. 지금은 그 기준을 넘었고, SQL·관심 차량 규칙·화면이 한곳에 있으므로 나누는 편이 맞습니다.

2 멀티페이지로 바꾸지 않는 이유

Streamlit에는 `st.navigation` 으로 페이지를 나누는 방법이 있습니다. 이 프로젝트에는 **맞지 않습니다**.

지금 화면은 한 페이지 안에서 탭으로 움직입니다.

- 1 공통 헤더
- 2 히어로 배너
- 3 관심 차량 버튼 (탭과 같은 줄)
- 4 차량 조회 / 차종 비교 / 도움말 탭

관심 차량 목록은 조회와 비교가 같은 `session_state` 를 씁니다. 페이지를 나누면 헤더·배너·관심 버튼을 각 페이지에 다시 넣어야 하고, 지금 맞춘 한 줄 배치가 깨지기 쉽습니다.

✅ 결론

목표는 "사이트를 여러 주소로 쪼개는 것"이 아니라, **같은 탭 구조를 유지한 채 코드 파일만 역할별로 나누는 것**입니다.

3 권장 폴더 구조

```

app/
  streamlit_app.py      # 설정, 공익광고, 헤더, 탭만 조립
  config.py             # 경로, URL, 제조사 아이콘·약칭
  db.py                 # SQLite 연결, SQL, read_query
  images.py             # 차량 사진, 아이콘, data URI
  interest.py           # 관심 차량 추가·삭제·식별
  styles.css            # 지금 markdown 안에 있는 CSS
  ui/
    chrome.py           # 헤더, 히어로, 관심 목록, 공익광고
    search.py           # 차량 조회 + 결과 4분할
    compare.py          # 차종 비교
    help.py             # 도움말
    components.py       # 링크 카드, 리포트 제목, 표 HTML

```

실행 방법은 지금과 같습니다. 프로젝트 루트에서:

```
streamlit run app/streamlit_app.py
```

`streamlit_app.py` 이름은 Streamlit 기본 엔트리 이름이므로 유지합니다.

4 각 파일이 맡는 일

4.1 `streamlit_app.py` — 조립만

화면의 "목차" 역할을 합니다. 대략 이런 순서만 남깁니다.

- 1 페이지 설정
- 2 CSS 로드
- 3 세션·공익광고
- 4 헤더 / 히어로 / 관심 차량
- 5 탭 세 개에 조회, 비교, 도움말 함수 연결

버튼·표·그래프를 이 파일에 다시 늘리지 않습니다.

4.2 `config.py` — 바뀌지 않는 값

- DB, 이미지, 아이콘, 배너 경로
- 정부 리콜센터 URL, 제조사 공식 사이트
- 제조사 짧은 이름, 아이콘 파일명

링크나 경로를 고칠 때 화면 파일을 열 필요가 없습니다.

4.3 `db.py` — 데이터만

- SQLite 연결 (`@st.cache_resource`)
- `read_query` (`@st.cache_data`)
- `MANUFACTURERS_SQL` , `RECALLS_SQL` 같은 조회문
- 요약 숫자처럼 **화면에 그리지 않는** 집계

SQL과 Streamlit 위젯을 섞지 않는 것이 핵심입니다.

4.4 `images.py` / `interest.py` — 재사용 규칙

- 차량 사진 찾기, 아이콘을 data URI로 바꾸기
- 관심 차량 최대 5대, 같은 차 중복 등록 막기, 태그 문구

이 함수들은 조회 탭과 비교 탭이 같이 씁니다. UI 파일에 복사하지 않습니다.

4.5 `styles.css` — 모양만

지금 `st.markdown("""<style>...</style>""")` 안에 있는 CSS를 파일로 옮깁니다. 줄 수가 가장 많이 줄어드는 단계입니다.

4.6 `ui/` — 화면만

파일	화면
<code>chrome.py</code>	모든 탭 위에 공통으로 보이는 부분
<code>search.py</code>	차량 조회, 결과 4분할
<code>compare.py</code>	차종 비교
<code>help.py</code>	FAQ
<code>components.py</code>	공식 사이트 카드, 리포트 제목처럼 두 탭이 같이 쓰는 조각

`st.button`, `st.metric`, `st.tabs` 같은 명령은 여기에만 둡니다.

5 나누는 기준 (한 줄로)

🎯 핵심 원칙

데이터와 화면을 분리한다.

- SQL, 숫자, 관심 차량 규칙 → `db.py` / `interest.py`
- 버튼, 카드, 그래프 → `ui/`

이렇게 하면 나중에 "누적 소비자 결함 신고수" 같은 문구만 바꿀 때 화면 레이아웃을 건드릴 일이 줄어듭니다.

파일을 너무 잘게 쪼개지는 않습니다. 함수 하나마다 파일을 만들지 않고, **한 탭 = 한 파일** 정도면 팀 작업에 충분합니다.

6 Streamlit에서 주의할 점

학습용으로 자주 나오는 실수이므로 같이 적어 둡니다.

⚠️ 1. `if __name__ == "__main__":` 으로 앱을 감싸지 않는다

Streamlit은 사용자가 위젯을 누를 때마다 파일 전체를 다시 실행합니다. 일반 파이썬 스크립트의 `main()` 패턴과 다릅니다.

⚠️ 2. 캐시는 데이터 함수에만 둔다

`@st.cache_data` 는 `read_query`, 이미지 별칭처럼 무거운 읽기에 둡니다. `render_dashboard()` 같은 화면 함수에는 씌우지 않습니다.

⚠ 3. import만으로 DB를 열지 않게 한다

지금은 파일 위쪽에서 제조사 목록을 바로 읽습니다. 나중 뒤에는 `db.py` 를 import하는 것만으로 연결이 열리지 않게 하고, 엔트리에서 한 번 호출하는 편이 안전합니다.

⚠ 4. 레거시 `pages/` 폴더는 쓰지 않는다

나중에 정말 멀티페이지가 필요하면 `app_pages/` + `st.navigation` 을 씁니다. 지금은 탭을 유지합니다.

7 작업 순서 제안

한 번에 다 옮기지 않습니다. 옮길 때마다 `streamlit run app/streamlit_app.py` 로 화면이 그대로인지 확인합니다.

순서	할 일	확인
1	CSS → <code>styles.css</code>	탭, 배너, 관심 버튼 위치가 같은지
2	상수·SQL·DB → <code>config.py</code> , <code>db.py</code>	제조사·차종 조회가 되는지
3	이미지·관심 차량 헬퍼	관심 등록, 사진, 아이콘
4	조회 / 비교 / 도움말 UI	세 탭 내용이 이전과 같은지
5	<code>streamlit_app.py</code> 는 조립만 남기기	실행 명령이 그대로인지

1단계만 해도 파일이 크게 짧아지고, 이후 단계는 서로 독립적으로 나눠 맡을 수 있습니다.

8 나누지 않아도 되는 것

모듈화가 "코드를 짧게 보이게"만 하는 작업이 되어서는 안 됩니다.

- 동작하는 조회·비교 로직을 다시 짜지 않습니다.
- 탭 UX, 관심 차량 5대 제한, SQL 결과 의미는 유지합니다.
- 파일을 나누는 것과 기능을 추가하는 것을 한 컷에 섞지 않습니다.

기능 추가가 있으면 모듈화 **이후**에, 해당 UI 파일만 고칩니다.

9 정리

- 한 파일은 이미 학습용 프로토타입 범위를 넘었습니다.
- 탭은 유지하고, 파일만 역할별로 나눕니다.
- CSS → DB/SQL → 헬퍼 → 탭 UI 순서로 옮깁니다.
- `streamlit_app.py` 는 "무엇을 어떤 순서로 보여 주는가"만 남깁니다.

실제 분리에 착수할 때는 이 문서의 7번 순서대로 진행하면 됩니다.

본 문서는 학습 및 팀 공유를 위한 구조 개선 제안이며, 현재 단계에서는 제안만 담고 실제 분리 작업은 진행하지 않았습니다.